

FMC 2026

Resolver os problemas certos em Software

Sumário

1. O melhor programador não é necessariamente quem escreve mais código	3
2. Software deve atender Driver e Estratégia	3
3. "Só porque podemos fazer, não significa que devemos"	4
4. IA é excelente para prototipagem – mas existe um limite.....	4
Código experimental.....	5
Código crítico de competição	5
5. Construir ou reutilizar?.....	5
Quanto mais genérico e estável o problema → mais sentido reutilizar. 5	
Quanto mais específico do robô → mais sentido a equipe dominar.	5
6. O exemplo interessante do PathPlanner	6
7. Não tenha medo de jogar código antigo fora	6
8. Configuração deve estar no código	7
9. Armazene valores de engenharia, não números mágicos.....	8
10. Software não conserta robô mal projetado	8
11. Encoder interno ou CANcoder externo?.....	9
12. Homing não precisa necessariamente de fim de curso	9
13. Menos sensores pode significar maior confiabilidade	10
14. Current Limit deveria ser obrigatório	10
15. Logs não são somente para quando alguma coisa quebra	11
16. Automação deve reduzir carga cognitiva do driver	11
17. Se a automação atrapalha o driver, remova	12
18. Validação deve acontecer de baixo para cima	12

19. Planeje também como o robô vai falhar	13
20. Visão não precisa significar Field Pose Estimation	13
21. Às vezes TX/TY resolve o problema	14
22. Câmera também é simplesmente um sensor	14
Conclusão	15
Principal mensagem da palestra	15

1. O melhor programador não é necessariamente quem escreve mais código

A principal mensagem da palestra é:

Antes de procurar uma solução, tenha certeza de que você identificou o problema correto.

Eles citam o chamado **Problema XY**: você possui o problema X, imagina que a solução seja Y e passa a perguntar como implementar Y, esquecendo de investigar X.

Um exemplo FRC apresentado foi:

Sintoma: estamos errando Coral no Reef.

Conclusão imediata: precisamos melhorar a visão.

Mas será?

Pode ser: **folga mecânica → erro de posição do braço → Coral errado.**

Ou: **elevador inconsistente → posição final incorreta → Coral errado.**

Ou ainda: **controle/PID mal ajustado → mecanismo oscila → Coral errado.**

Nesse cenário, investir horas melhorando AprilTags não resolve o verdadeiro problema.

2. Software deve atender Driver e Estratégia

Gostei bastante dessa definição da palestra:

O "cliente" da equipe de software é o Driver Team e a Estratégia.

Isso muda bastante a forma de pensar programação.

Não basta perguntar: "O intake funciona?"

A pergunta deveria ser: "Esse intake funciona da maneira que o driver precisa para maximizar o desempenho do robô?"

Por isso eles defendem conversar constantemente com os drivers e iterar os controles.

Inclusive, antes de começar a programar, eles criam um **documento vivo de arquitetura/funcionalidades**, mostrando o que o robô deverá fazer e como determinadas automações funcionarão. O documento também funciona como rastreador de tarefas e fonte de verdade durante a temporada.

3. "Só porque podemos fazer, não significa que devemos"

Essa frase aparece diversas vezes implicitamente na palestra.

Imagine que conseguimos implementar:

Shoot on the Move.

Legal.

Mas precisamos perguntar:

É consistente?

Marca mais pontos que simplesmente parar e atirar?

Se o sistema sofisticado acerta 75% das bolas e o sistema simples acerta 98%, talvez o segundo seja competitivo e o primeiro seja apenas tecnicamente impressionante.

Essa é uma distinção importante:

Complexidade técnica $\neq$ desempenho competitivo.

A equipe deve avaliar cada funcionalidade pelo benefício que realmente entrega em campo.

4. IA é excelente para prototipagem – mas existe um limite

A discussão sobre IA foi particularmente interessante.

Segundo eles, IA/programação assistida é muito boa para conseguir rapidamente uma solução **"90% pronta"**.

Eles usam para:

- protótipos no começo da temporada;
- ferramentas internas;
- tuning;
- debugging;
- análise de partidas;
- explorar rapidamente ideias;
- experimentar mudanças grandes.

Isso reduz muito o custo de testar uma ideia antes de decidir investir nela.

Mas eles fazem uma distinção importante.

Código experimental

IA pode acelerar bastante.

Código crítico de competição

Precisa ser: **entendido → revisado → testado → validado → depurável.**

Eles dão um exemplo propositalmente extremo:

Imagine estar na **Final 3 do Championship**, o código falhar e ninguém da equipe saber explicar por quê porque grande parte dele foi gerada sem compreensão real do sistema.

Por isso a posição apresentada não é "não use IA".

É: **Não coloque no robô código crítico que sua equipe não consegue entender e depurar.**

5. Construir ou reutilizar?

Esse ponto conversa diretamente com nossas discussões sobre **YAGSL**, **PathPlanner**, **Phoenix/CTRE etc.**

A palestra propõe basicamente pensar:

Quanto mais genérico e estável o problema → mais sentido reutilizar.

Exemplo: **Swerve**

Não existe grande vantagem em uma equipe iniciante reconstruir absolutamente tudo do zero apenas porque consegue.

Frameworks existentes já resolveram muitos problemas e foram testados extensivamente.

Quanto mais específico do robô → mais sentido a equipe dominar.

Por exemplo:

ShooterSubsystem

IntakeSubsystem

ElevatorSubsystem

Criar uma abstração universal de "qualquer shooter" pode acabar tornando o código mais complexo que simplesmente implementar aquele mecanismo.

6. O exemplo interessante do PathPlanner

A própria equipe da palestra criou seu framework autônomo chamado **Trailblazer**.

Mas eles fazem questão de dizer:

A maioria das equipes provavelmente não deveria fazer isso.

Eles utilizaram PathPlanner durante alguns anos.

Chegaram, porém, a um ponto em que aproximadamente **90% das necessidades eram atendidas**, enquanto os 10% restantes eram difíceis de alcançar porque exigiam comportamentos muito específicos.

Só então desenvolveram a própria solução.

Isso apresenta uma ótima regra:

Não desenvolva sua própria biblioteca porque é divertido desenvolver uma biblioteca.

Faça isso quando:

a solução existente realmente se tornou uma limitação competitiva.

7. Não tenha medo de jogar código antigo fora

Esse ponto é excelente para FRC.

Em empresas, existem sistemas com: **10, 20, 30 anos de código**.

Na FRC, o jogo muda todo ano.

Portanto, eles defendem aproveitar essa oportunidade.

Código específico de: **Coral, Algae, Shooter, Intake, Elevator etc.**

pode simplesmente desaparecer quando não for mais necessário.

Eles mantêm principalmente abstrações realmente estáveis:

- ferramentas matemáticas;
- estrutura de autos;
- algumas abstrações de subsistemas;
- utilidades;
- infraestrutura;
- Swerve.

O restante pode começar novamente.

E existe uma regra particularmente importante:

Se existe código crítico no robô que ninguém da equipe entende, existe um problema.

Ou a equipe aprende e assume aquele código, ou considera removê-lo/simplificá-lo.

8. Configuração deve estar no código

Essa é uma prática que eu adotaria como padrão.

Imagine trocar um Talon FX durante uma competição.

Depois da troca alguém precisa lembrar:

- **CAN ID**
- **Inverted**
- **Current Limit**
- **Neutral Mode**
- **PID**
- **Gear Ratio**
- **Sensor Ratio**
- **Ramp Rate**
- **Motion Magic...**

Se isso depende de alguém abrir uma ferramenta e clicar manualmente nas configurações, você criou oportunidade para erro humano.

A proposta é: **o código configura o hardware.**

Assim, quando o robô inicializa: `motor.configurator.apply(config);`

O dispositivo volta automaticamente para um **estado conhecido**.

Além disso, Git passa a registrar mudanças de configuração.

9. Armazene valores de engenharia, não números mágicos

Esse ponto também é muito bom.

Em vez de escrever algo como: `double ratio = 119.35;`

e seis meses depois ninguém saber de onde saiu 119.35, mantenha no código os elementos físicos que geraram aquele valor.

Por exemplo:

```
double motorGear = 10;
double mechanismGear = 48;

double gearRatio = mechanismGear / motorGear;

Agora o mentor de mecânica diz:

"Mudamos a engrenagem de 10 para 12 dentes."

O programador simplesmente muda:

double motorGear = 12
```

Isso reduz a distância entre **CAD/Mecânica ↔ Software**.

10. Software não conserta robô mal projetado

Talvez essa seja uma das mensagens mais importantes da apresentação.

Você não consegue compensar indefinidamente com software um robô mecanicamente ruim.

Um elevador está escorregando.

A equipe pergunta: "Como resolvemos isso no software?"

Pode até ser possível diminuir aceleração, limitar corrente ou compensar alguma coisa.

Mas talvez o problema verdadeiro seja: **o elevador não deveria estar escorregando.**

A solução de software pode apenas mascarar o problema e ainda reduzir o desempenho do mecanismo.

11. Encoder interno ou CANcoder externo?

Essa discussão foi particularmente interessante.

A filosofia deles é:

Se o encoder interno do motor resolve, use o encoder interno.

Isso elimina:

- sensor;
- suporte;
- cabo;
- conector;
- configuração;
- calibração;
- possíveis pontos de falha.

Eles afirmam que conseguem utilizar encoder interno na maioria dos mecanismos, reservando sensores externos para casos em que realmente são necessários.

12. Homing não precisa necessariamente de fim de curso

Outra ideia interessante:

Um elevador pode descer lentamente até encontrar o limite físico.

O software observa, por exemplo: **corrente ↑ + velocidade ≈ 0**

e conclui: Cheguei ao zero.

Então: posição atual = 0

Dessa forma, dependendo do mecanismo, é possível eliminar o limit switch.

Mas existe uma condição importante: **isso precisa ser desenvolvido e testado de forma controlada.**

Eles descrevem um processo em camadas, começando com PID, tensões e demais parâmetros em condições seguras e adicionando comportamento progressivamente.

13. Menos sensores pode significar maior confiabilidade

Isso resume uma filosofia recorrente da palestra:

Não adicione complexidade sem necessidade.

Se o motor já consegue fornecer: **posição + velocidade + corrente**

talvez não seja necessário adicionar: **encoder externo + limit switch.**

Mas existem casos em que o encoder externo é necessário.

Um exemplo citado é um mecanismo de escalada com corda, porque a relação entre rotação do motor e posição real pode não ser determinística. Nesse caso, o sensor externo mede melhor aquilo que realmente importa.

Portanto, não é: **"encoder externo é ruim".**

É: **"qual informação realmente precisamos medir?"**

14. Current Limit deveria ser obrigatório

Essa parte eu destacaria bastante para nossas equipes.

A palestra diferencia:

Supply Current Limit → protege principalmente o sistema elétrico/bateria.

Stator Current Limit → protege principalmente motor/mecanismo.

E a recomendação apresentada é forte:

Configure limites de corrente em todos os motores.

Eles relatam encontrar frequentemente em competições robôs com:

brownout, queda de tensão, disjuntor principal abrindo e drivetrain precisando rodar lentamente, quando o verdadeiro problema está relacionado ao gerenciamento de corrente.

Também destacam algo extremamente simples:

verifique os cabos da bateria antes das partidas.

15. Logs não são somente para quando alguma coisa quebra

Eles possuem uma prática muito interessante:

Depois de cada dia de competição, analisam os logs independentemente de terem percebido algum problema.

Ou seja:

não espere falhar para investigar.

Eles procuram verificar se:

- correntes estão normais;
- tensões estão normais;
- mecanismos estão dentro das especificações;
- comportamentos inesperados apareceram;
- algum componente está começando a degradar.

Isso conversa muito bem com AdvantageKit e seria uma ótima rotina para implementarmos.

16. Automação deve reduzir carga cognitiva do driver

Eles mostram um exemplo em que determinada sequência exigia aproximadamente **oito comandos/botões**.

Posteriormente, automações permitiram realizar essencialmente a mesma sequência utilizando cerca de **quatro comandos**.

Isso é um ótimo objetivo para programação:

Driver decide O QUE fazer.

Software decide COMO executar.

Mas existe uma condição.

A automação precisa ser:

natural + consistente + resistente a falhas + competitivamente útil.

17. Se a automação atrapalha o driver, remova

Eles desenvolveram uma assistência automática para atravessar determinada região do campo.

No papel: **excelente ideia.**

Na prática, o sistema ativava em situações indesejadas e interferia na pilotagem.

Resultado?

Desativaram.

O driver conseguia fazer tão bem quanto o software.

Essa decisão é importante porque existe uma tendência natural de pensar:

"Gastamos muito tempo desenvolvendo isso, então precisamos usar."

Não.

Se não melhora o robô: **delete.**

18. Validação deve acontecer de baixo para cima

Esse talvez seja o ensinamento técnico mais importante da palestra.

Não comece testando: "Nosso Auto de 6 peças funciona?"

Comece testando: **1 roda → módulo Swerve → drivetrain → odometria → path following → mecanismos → sequenciamento → Auto completo.**

Eles dão inclusive um exemplo aparentemente bobo:

verifique o gear ratio antes de ajustar PID.

Um erro pequeno na relação de transmissão propaga erro por todo o sistema.

Quando finalmente o Auto falha, fica extremamente difícil descobrir onde nasceu o problema.

Isso é exatamente o tipo de metodologia que eu transformaria em **checklist padrão de programação dos nossos times.**

19. Planeje também como o robô vai falhar

Quanto mais automação adicionamos, maior a quantidade de modos de falha.

Então eles perguntam:

E se o sensor morrer?

E se o intake quebrar?

E se perdermos determinado motor?

Nem toda falha precisa de uma solução sofisticada.

Às vezes: Intake quebrou → não pontuamos mais → vamos defender.

Mas algumas falhas podem justificar:

Manual Override

Disable Sensor

Disable Automation

Fallback Mode

E existe uma observação essencial:

o modo de emergência também precisa ser testado.

Não adianta descobrir na semifinal que o "Manual Override" não funciona porque ninguém o testou durante três meses.

20. Visão não precisa significar Field Pose Estimation

Essa parte eu achei particularmente interessante.

Existe uma tendência na FRC de: **Visão = AprilTags + Pose Estimation completa do campo.**

Eles questionam isso.

Field Pose Estimation envolve muitas fontes de erro: **posição da câmera → calibração → AprilTag → geometria do campo → múltiplas câmeras → odometria → fusão dos dados.**

Talvez você não precise de tudo isso.

21. Às vezes TX/TY resolve o problema

Se precisamos alinhar o robô com um objeto que a câmera consegue enxergar claramente, podemos simplesmente utilizar o erro visual.

Conceitualmente:

TX = distância horizontal do alvo ao centro da imagem

TX > 0 → girar para direita

TX < 0 → girar para esquerda

TX ≈ 0 → alinhado

É muito mais simples que reconstruir toda a posição do robô no campo.

A ideia é:

Utilize a menor quantidade de informação necessária para resolver o problema.

A palestra apresenta exemplos de equipes utilizando Limelights dessa forma para alinhamento direto com AprilTags/objetos.

22. Câmera também é simplesmente um sensor

Gostei bastante dessa mudança de mentalidade.

Em 2025, eles precisavam centralizar o Coral dentro do mecanismo.

Uma possível solução mecânica seria adicionar: **rodas de centralização + motores + transmissão + peso.**

Mas já existia uma Limelight olhando para aquela região.

Então utilizaram: **Color Pipeline → posição do Coral → software ajusta o mecanismo.**

Segundo eles, a solução foi implementada em **menos de duas horas** e permaneceu sendo utilizada.

Isso mostra algo muito interessante: **visão não precisa servir exclusivamente para AprilTags.**

Conclusão

Dessa palestra eu transformaria principalmente estes pontos em padrão de desenvolvimento:

- 1. Criar um documento de arquitetura antes do código.** O que o robô precisa fazer, estados dos mecanismos, automações e interação do driver.
- 2. Perguntar sempre "qual problema estamos tentando resolver?"** Principalmente antes de adicionar sensores ou automações complexas.
- 3. Tratar Driver Team como cliente do software.** Fazer revisão periódica dos controles com o driver.
- 4. Current Limit obrigatório em todos os motores.**
- 5. Configuração de Talon FX/Spark MAX no código.** Evitar configurações manuais que precisam ser refeitas após substituição.
- 6. Reduzir sensores desnecessários.** Utilizar encoder integrado quando suficiente e externo quando existir justificativa técnica.
- 7. Criar rotina padronizada de homing.**
- 8. Validar de baixo para cima.** Motor → mecanismo → subsystem → integração → automação.
- 9. Analisar logs depois de cada competição.** Não apenas quando houver falha.
- 10. Criar modos de contingência para funções realmente críticas.**
- 11. Reutilizar software consolidado quando fizer sentido.** Não reinventar Swerve/path planning simplesmente para dizer que a equipe desenvolveu tudo.
- 12. Não carregar dívida técnica desnecessária de uma temporada para outra.**
- 13. Utilizar IA principalmente para prototipagem, ferramentas e debugging, mas exigir compreensão e validação para código crítico de competição.**
- 14. Tratar câmera como sensor genérico.** AprilTags, cor, objetos, alinhamento relativo etc.
- 15. Avaliar toda automação pela pergunta: "isso realmente aumenta nosso desempenho?"**

Principal mensagem da palestra

Eu resumiria em:

Software bom na FRC não é o software que faz mais coisas. É o software que resolve os problemas certos, da maneira mais simples, confiável e compreensível possível.

E existe uma segunda mensagem igualmente importante:

Antes de tentar compensar um problema com código, descubra se o problema realmente pertence ao software.

Para a estrutura que estamos desenvolvendo para os times, essa palestra é especialmente útil porque vários desses pontos podem virar **procedimentos/checklists oficiais de programação**, principalmente configuração de motores, current limit, homing, validação em camadas, logs e critérios para automação.